KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 7 Quiz

React III: API Tüketimi ve Yönlendirme

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (6 puan)

4 • Değerlendirme Hedef 1

`fetch(...)` çağrısının döndürdüğü `Promise` ne zaman reddedilir (*reject*), ne zaman normal biçimde çözülür (*resolve*)? Sunucunun `404/500` gibi bir HTTP durum kodu döndürdüğü durumu da göz önünde bulundurarak doğru açıklamayı seçiniz.

(a) `Promise` her zaman durum kodu 200–299 aralığı dışında olduğunda reddedilir; sunucunun 404 veya 500 dönmesi bile bu duruma göre bir `Promise` reddi sayılır.

(b) `fetch` hiçbir zaman reddedilmez; sunucuya hiç ulaşılamasa bile `Promise` normal biçimde çözülür, yalnızca cevap gövdesi boş kalır, `response.status` `0` olur.

(c) `Promise` yalnızca bir ağ hatasında (sunucuya hiç ulaşılamadı) reddedilir; sunucu 404/500 gibi bir HTTP cevabı üretse bile `Promise` normal biçimde çözülür, yalnızca `response.ok` `false` olur.

(d) `Promise` yalnızca `response.json()` çağrısı başarısız olduğunda (gövde JSON değilse) reddedilir; durum kodunun kendisi `Promise`'in sonucunu hiç etkilemez, çünkü `fetch` yalnızca gövdenin biçimini denetler.

Soru 2 (6 puan)

4 • Değerlendirme Hedef 1

`apiGet` fonksiyonunda `const response = await fetch(...)` satırından sonra neden `await response.json()` biçiminde ikinci bir `await` gerekir; tek bir `await fetch(...)` yeterli olmaz mı?

(a) `response.json()` metodunun kendisi de bir `Promise` döndürür; gövdenin ayrıştırılması ayrı bir asenkron adımdır, bu yüzden ikinci bir `await` gerekir.

(b) `fetch` zaten gövdeyi de ayrıştırır; ikinci `await` yalnızca TypeScript'in zorunlu kıldığı bir sözdizimi kuralıdır, çalışma zamanında hiçbir etkisi yoktur.

(c) İkinci `await` yalnızca hata ayıklama (*debug*) amaçlıdır; kaldırılırsa kod tamamen aynı biçimde çalışmaya devam eder, yalnızca konsol çıktısı değişir.

(d) `response.json()` Senkron çalışır; `await` eklenmesi yalnızca bir stil tercihidir, sonucu değiştirmez, TypeScript de bu satırı hiç denetlemez.

Soru 3 (8 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki kesitler `src/api/etkinlikler.ts` ve `src/api/koltuklar.ts` dosyalarından alınmıştır.

```ts
// GET /etkinlikler → Etkinlik[]; mekan verilirse sunucu tarafında filtrelenir.
export function etkinlikleriGetir(
  mekan?: string,
  signal?: AbortSignal,
): Promise<Etkinlik[]> {
  const sorgu = mekan ? `?mekan=${encodeURIComponent(mekan)}` : '';
  return apiGet<Etkinlik[]>(`/etkinlikler${sorgu}`, { signal });
}

// GET /etkinlikler/{id} → Etkinlik; bulunamazsa apiGet bir ApiHatasi (404) fırlatır.
export function etkinlikGetir(id: number, signal?: AbortSignal): Promise<Etkinlik> {
  return apiGet<Etkinlik>(`/etkinlikler/${id}`, { signal });
}
```

```ts
// GET /etkinlikler/{etkinlikId}/koltuklar → Koltuk[]; durum verilirse
// sunucu tarafında filtrelenir (ör. yalnızca boş koltuklar).
export function koltuklariGetir(
  etkinlikId: number,
  durum?: KoltukDurumu,
  signal?: AbortSignal,
): Promise<Koltuk[]> {
  const sorgu = durum ? `?durum=${durum}` : '';
  return apiGet<Koltuk[]>(`/etkinlikler/${etkinlikId}/koltuklar${sorgu}`, {
    signal,
  });
}

// PATCH /koltuklar/{id} — yalnızca durum alanını günceller; koltuk başka
// biri tarafından zaten alınmışsa sunucu bir 404/409 dönebilir (bu
// dosyanın kendisi bunu YAKALAMAZ, çağıran kod — EtkinlikDetaySayfasi —
// ApiHatasi'nı yakalayıp iyimser değişikliği geri alır).
export function koltukGuncelle(id: number, durum: KoltukDurumu): Promise<Koltuk> {
  return apiPatch<{ durum: KoltukDurumu }, Koltuk>(`/koltuklar/${id}`, { durum });
}
```

(a) `koltuklariGetir(3, 'bos')` çağrısı sunucuya tam olarak hangi URL'e (yol + sorgu dizesi) bir `GET` isteği gönderir? (b) `etkinlikGetir(999)` çağrısı (var olmayan bir etkinlik id'si) çalıştırıldığında ne olur — döndürdüğü `Promise` reddedilir mi, hangi sınıftan, hangi `status` değeriyle bir hata fırlatılır?

Soru 4 (10 puan)

4 • Değerlendirme Hedef 1

Hf.6'da alınan camelCase kararı, bu haftanın `apiGet<Etkinlik[]>('/etkinlikler')` çağrısının dönüş tipini nasıl etkiler; Hf.6'nın sunucusu bunun yerine snake_case JSON döndürseydi `src/api/` katmanında ne değişirdi? `TABAN_URL` sabitinin `src/api/istemci.ts`'te tek bir yerde tutulmasının amacını da açıklayınız.

Soru 4 (devamı)

Soru 5 (6 puan)

4 • Değerlendirme Hedef 2

`VeriDurumu<T>` tipi ortak bir `tur` alanıyla ayırt edilen bir *discriminated union*'dır. `durum.tur === 'yukleniyor'` İken `durum.veri`'ye erişmeye çalışırsanız ne olur?

(a) Çalışma zamanında `undefined` döner; derleme (*compile*) sırasında hiçbir hata verilmez, TypeScript bu erişimi sessizce geçer.

(b) TypeScript derleme hatası verir — `'yukleniyor'` dalı `veri` alanı içermez; erişim ancak `durum.tur === 'basarili'` kontrolüyle güvenlidir.

(c) TypeScript union'daki tüm dalların ortak bir `veri` alanına sahip olduğunu varsayar, erişim her zaman izin verilir, tip daraltmaya gerek kalmaz.

(d) Yalnızca `strict` modu kapalıyken bu bir hata sayılır; açıkken sorun çıkarmaz, çünkü `strict` modu yalnızca null/undefined denetimini etkiler.

Soru 6 (6 puan)

4 • Değerlendirme Hedef 2

`useEffect(() => { ... }, bagimliliklar)` İçine doğrudan bir `async` fonksiyon verilemez; bunun yerine efekt içinde ayrı bir `async` fonksiyon tanımlanıp hemen çağrılır (`useVeri`'deki `veriyiGetir`). Bunun nedeni nedir?

(a) `async` fonksiyonlar `useEffect` içinde yasaktır; JavaScript motoru bu kodu hiç çalıştırmaz, derleme aşamasında bir sözdizimi hatası verir.

(b) Performans nedeniyle; `async` fonksiyonlar `useEffect`'i gözle görülür biçimde yavaşlatır, her *render*'da yeniden oluşturulmaları gerekir.

(c) Yalnızca TypeScript tip denetimi buna izin vermez; salt JavaScript'te hiçbir sorun olmaz, derleme sonrası kod aynen çalışır.

(d) `useEffect`'in geri çağırım fonksiyonu ya hiçbir şey ya da bir temizleme fonksiyonu döndürmelidir; bir `async` fonksiyon her zaman bir `Promise` döndürür — React bunu temizleme sanıp çağırır.

Soru 7 (8 puan)

4 • Değerlendirme Hedef 2

Aşağıdaki kesit `src/hooks/useVeri.ts` dosyasından alınmıştır.

```
// Ortak bir 'tur' alanıyla ayırt edilen, her dalın FARKLI ek alanlar
// taşıdığı bir discriminated union: durum.tur === 'basarili' kontrolünden
// SONRA durum.veri'ye erişmek güvenlidir, 'yukleniyor' dalında durum.veri
// HİÇ YOKTUR — TypeScript bunu derleme zamanında denetler.
export type VeriDurumu<T> =
```

Soru 7 (devamı)

```
  | { tur: 'yukleniyor' }
  | { tur: 'hata'; mesaj: string }
  | { tur: 'basarili'; veri: T };
```

```
// cagri, AbortController'ın signal'ini ALAN ve bir Promise<T> DÖNDÜREN bir
// fonksiyondur (ör. (signal) => etkinlikleriGetir(undefined, signal)).
// bagimliliklar değiştiğinde (ör. yeni bir :id) efekt YENİDEN çalışır;
// önceki istek controller.abort() ile İPTAL edilir — bu sayede yarış
// durumunda (eski cevap yeni cevaptan GEÇ gelirse) state artık ESKİ
// veriyle GÜNCELLENMEZ.
export function useVeri<T>(
  cagri: (signal: AbortSignal) => Promise<T>,
  bagimliliklar: unknown[],
): VeriDurumu<T> {
  const [durum, setDurum] = useState<VeriDurumu<T>>({ tur: 'yukleniyor' });

  useEffect(() => {
    const controller = new AbortController();
    setDurum({ tur: 'yukleniyor' });

    // useEffect'in kendisi async OLAMAZ (bir Promise döndüremez); bu
    // yüzden efekt içinde AYRI bir async fonksiyon tanımlanıp HEMEN
    // çağrılır.
    async function veriyiGetir() {
      try {
        const veri = await cagri(controller.signal);
        setDurum({ tur: 'basarili', veri });
      } catch (hata) {
        if (hata instanceof DOMException && hata.name === 'AbortError') {
          // İstek bilinçli olarak iptal edildi (yeni bağımlılık ya da
          // unmount) — state'i GÜNCELLEMEDEN sessizce çık.
          return;
        }
        const mesaj =
          hata instanceof ApiHatasi
            ? typeof hata.detail === 'string'
              ? hata.detail
              : hata.message
            : hata instanceof Error
              ? hata.message
              : 'Bilinmeyen bir hata oluştu.';
        setDurum({ tur: 'hata', mesaj });
      }
    }

    veriyiGetir();

    return () => controller.abort();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, bagimliliklar);

  return durum;
}
```

(a) `AbortController`'ın `signal`'i `cagri` fonksiyonuna nasıl geçirilir; `controller.abort()` ne zaman (hangi iki durumda) çağrılır? (b) `hata instanceof DOMException && hata.name === 'AbortError'` kontrolü olmasaydı, bir istek iptal edildiğinde (ör. hızlı sayfa değişimin-

Soru 7 (devamı)

de) kullanıcı ekranda ne görürdü?

Soru 8 (10 puan)

4 • Değerlendirme Hedef 2

`EtkinlikDetaySayfasi`'nda kullanıcı `/etkinlikler/3`'ten `/etkinlikler/7`'ye hızlıca geçtiğinde iki ayrı istek yola çıkar. `useVeri` İçindeki `AbortController` bu senaryoda yarış durumunu (*race condition*) nasıl önler; bunu iki ayrı gişeye art arda gönderilen iki mektup benzetmesini kullanarak açıklayınız.

Soru 9 (6 puan)

4 • Değerlendirme Hedef 3

`http://localhost:5173` ile `http://127.0.0.1:8000` iki farklı *origin* (köken) midir? Yalnızca port farkı bile yeterli midir?

(a) Evet, iki farklı *origin*'dir; *origin* protokol+alan adı+port üçlüsüdür, yalnızca port farkı bile farklı *origin* sayılması için yeterlidir.

(b) Hayır, aynı *origin*'dir; çünkü ikisi de `localhost`/`127.0.0.1` üzerinde çalışır, tarayıcı ikisini de yerel makine sayar.

(c) Yalnızca protokol farkı (`http`/`https`) *origin* farkı oluşturur, port farkı önemsizdir, tarayıcı portu göz ardı eder.

(d) *Origin* yalnızca alan adına bakar; IP adresi kullanan adresler *origin* kontrolüne tabi değildir, doğrudan izinli sayılır.

Soru 10 (6 puan)

4 • Değerlendirme Hedef 3

`Content-Type: application/json` gövdeli bir `POST /biletler` isteği gönderildiğinde tarayıcı neden önce bir `OPTIONS` isteği gönderir?

(a) Her `fetch` çağrısı, metottan bağımsız olarak her zaman önce bir `OPTIONS` isteği gönderir, bu tarayıcının sabit davranışıdır.

(b) `OPTIONS` isteği yalnızca sunucu HTTPS kullanmadığında gönderilir; HTTPS'li sunucularda *preflight* hiç tetiklenmez.

(c) `Content-Type: application/json` başlıklı istek "basit olmayan" sayılır; tarayıcı asıl istekten önce bir `OPTIONS` ile izin sorar (*preflight*).

Soru 10 (devamı)

(d) `OPTIONS` isteği istemci tarafında yalnızca bir hata ayıklama aracı tarafından eklenir; üretimde hiç gönderilmez, yalnızca geliştirme ortamına özgü bir davranıştır.

Soru 11 (8 puan)

4 • Değerlendirme Hedef 3

Aşağıdaki kesitler `app/main.py` ve `tests/test_cors.py` dosyalarından alınmıştır.

```python
# Bu haftanın TEK sunucu değişikliği: tarayıcı, Vite geliştirme
# sunucusunun (http://localhost:5173) origin'inden bu API'nin (8000)
# origin'ine fetch attığında varsayılan olarak cevabı JS koduna
# okutturmaz (aynı-origin politikası). CORSMiddleware, sunucunun
# cevaplarına "bu cevabı localhost:5173 okuyabilir" başlığını (ve
# gerektiğinde preflight OPTIONS cevabını) ekler. allow_origins bilinçli
# olarak yalnızca geliştirme adresini içerir — "*" (herkese izin) üretim
# ortamında güvenlik riski taşır (ayrıntı Hf.11).
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```python
def test_cors_izinli_origin(client):
    cevap = client.get(
        "/etkinlikler", headers={"Origin": "http://localhost:5173"}
    )
    assert cevap.status_code == 200
    assert cevap.headers["access-control-allow-origin"] == "http://localhost:5173"
```

```python
def test_cors_izinsiz_origin(client):
    cevap = client.get(
        "/etkinlikler", headers={"Origin": "http://localhost:4000"}
    )
    # İstek sunucuda yine de ÇALIŞIR (200 döner) — CORS bir sunucu
    # yetkilendirmesi DEĞİL, tarayıcının cevabı SAKLAMASINI sağlayan bir
    # başlık mekanizmasıdır; izinsiz origin'de yalnızca bu başlık EKSİK
    # olur, tarayıcı cevabı JS koduna ULAŞTIRMAZ.
    assert cevap.status_code == 200
    assert "access-control-allow-origin" not in cevap.headers
```

(a) `test_cors_izinsiz_origin` testinde `cevap.status_code == 200` doğrulanıyor, ama `access-control-allow-origin` başlığının yokluğu da doğrulanıyor. Bu, isteğin sunucuda çalışıp çalışmadığı konusunda ne anlatır — CORS bir sunucu yetkilendirmesi midir? (b) `allow_origins=["http://localhost:5173"]` Yerine `allow_origins=["*"]` yazılsaydı, aynı test (Origin: `http://localhost:4000` ile) HÂLÂ geçer miydi?

Soru 12 (10 puan)

4 • Değerlendirme Hedef 3

Tarayıcının aynı-*origin* politikası tam olarak neyi engeller (cevabı sunucuya ulaştırmama mı, yoksa cevabı JS koduna okutturmama mı)? Bu politikanın neden var olduğunu apartman kapıcısı benzetmesini kullanarak, bir güvenlik senaryosuyla açıklayınız.

Soru 12 (devamı)

Soru 13 (6 puan)

4 • Değerlendirme Hedef 4

Bilet formu başarıyla gönderildikten sonra `/biletlerim` sayfasına geçmek için `useNavigate` kullanılır, `<Link to="/biletlerim">` değil. Bu seçimin nedeni nedir?

(a) `Link` yalnızca statik sayfalara yönlendirebilir; dinamik/parametreli URL'lere (`?kullaniciAdi=...`) yönlendiremez, bu yüzden böyle durumlarda `useNavigate` gerekir.

(b) `Link`, kullanıcının tıklamasını gerektiren bir JSX bileşenidir; `useNavigate` ise bir olay işleyicisi içinden, programatik olarak tetiklenebilir.

(c) `Link` ve `useNavigate` aynı işi görür; aralarındaki seçim tamamen keyfidir, ikisi de her bağlamda birbirinin yerine kullanılabilir.

(d) `useNavigate` sayfayı tam yeniler (F5 gibi); `Link` ise SPA içi geçiş yapar, bu yüzden `useNavigate` daha güvenilir kabul edilir.

Soru 14 (8 puan)

4 • Değerlendirme Hedef 4

Aşağıdaki kesitler `src/App.tsx` ve `src/bilesenler/Duzen.tsx` dosyalarından alınmıştır.

```tsx
// Uygulamanın TAMAMI artık BrowserRouter içinde yaşar; Routes içindeki
// Route'lar SIRAYLA değerlendirilir, İLK eşleşen yol RENDER edilir.
// Route element={<Duzen />} İÇİNE yerleştirilen İÇ Route'lar (index,
// "etkinlikler/:id", "biletlerim") HEP AYNI Duzen nav çubuğunun ALTINDA
// görünür (Duzen'in <Outlet /> yer tutucusu SAYESİNDE); path="*" EN SONDA
// tanımlanır, tanımsız her adres için BulunamadiSayfasi render edilir.
export function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<Duzen />}>
          <Route index element={<EtkinliklerSayfasi />} />
          <Route path="etkinlikler/:id" element={<EtkinlikDetaySayfasi />} />
          <Route path="biletlerim" element={<BiletlerimSayfasi />} />
          <Route path="*" element={<BulunamadiSayfasi />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

Soru 14 (devamı)

```jsx
// Tüm sayfaların ORTAK çerçevesi: bir nav çubuğu (her zaman görünür) ve
// altında <Outlet /> — o ANDA eşleşen alt rotanın (EtkinliklerSayfasi,
// EtkinlikDetaySayfasi, BiletlerimSayfasi, BulunamadiSayfasi) render
// EDİLECEĞİ yer tutucu. Link, normal bir <a> etiketinin AKSİNE tıklandığında
// sunucuya YENİ bir HTTP isteği göndertmez — yalnızca adres çubuğunu
// güncelleyip eşleşen bileşeni render eder.
export function Duzen() {
  return (
    <div className="duzen">
      <nav className="duzen-nav">
        <span className="duzen-baslik">KTÜ Etkinlik ve Bilet Sistemi</span>
        <Link to="/">Etkinlikler</Link>
        <Link to="/biletlerim">Biletlerim</Link>
      </nav>
      <main>
        <Outlet />
      </main>
    </div>
  );
}
```

(a) `path="*"` rotası `<Route element=<Duzen />>` bloğunun içine, diğer rotalarla (`index`, `"etkinlikler/:id"`, `"biletlerim"`) aynı seviyede yerleştirilmiştir. Bunun sonucu olarak, tanımsız bir adrese (`/xyz`) gidildiğinde `BulunamadiSayfasi` NAV çubuğuyla birlikte mi, yoksa tek başına mı görünür? Nedenini açıklayın. (b) `<Route index element=<EtkinliklerSayfasi /> />` satırında `path` prop'u yoktur, yalnızca `index` vardır. Bu rota hangi URL'e karşılık gelir?

Soru 15 (10 puan)

4 • Değerlendirme **Hedef 4**

Bir `EtkinlikKarti`'ndeki `Link`'e tıklamanın, aynı URL'i adres çubuğuna elle yazıp Enter'a basmaktan nasıl farklı olduğunu; DevTools Network sekmesinde görülecek farkı ve bu farkın Hf.1'in SPA vs SSR ayrımıyla ilişkisini açıklayınız.